

Technikum nr 25

im. Stanisława Staszica

Temat: PokerKalk Techniczna

Antoni Paczkowski 4AP

Warszawa 28.05.2026

1. Wykorzystane Technologie i Biblioteki

2. Jak aplikacja przechowuje dane (Zmienne i Klasy)
 - 2.1. Dane z Internetu (Karty graficzne)
 - 2.2. Dane do Obliczeń (Logika gry)
 - 2.3. Pamięć obecnego stołu
3. Interfejs użytkownika
4. Komunikacja z Api
5. Mechanika Wyboru Kart
6. Silnik Logiczny: Ewaluacja Układów Pokerowych
7. Algorytm Obliczania Szans (Symulacja Monte Carlo)

Wstęp: Co robi aplikacja?

PokerKalk to mobilna aplikacja na system Android stworzona do symulacji i obliczania szans na wygraną w popularnej grze karcianej Poker Texas Hold'em.

Głównym zadaniem programu jest pomoc graczom w ocenie siły ich "ręki" (kart własnych) w stosunku do kart leżących na stole (Board). Aplikacja pozwala na:

- Wybór od 2 do 5 graczy biorących udział w rozdaniu.
- Rozdanie kart dla każdego z graczy oraz wyłożenie kart wspólnych na stół.
- Płynne pobieranie grafik kart przez Internet z zewnętrznej bazy danych.
- Obliczenie procentowej szansy na wygraną każdego z graczy na podstawie aktualnej sytuacji na stole, wykorzystując do tego metodę wielokrotnych, losowych symulacji.

Projekt łączy w sobie intuicyjny interfejs użytkownika z dość zaawansowanym algorytmem oceny układów pokerowych

1. Wykorzystane Technologie i Biblioteki

Aplikacja została napisana w języku Kotlin z wykorzystaniem systemu widoków (XML) dla systemu Android. Cała logika działania mieści się w głównej aktywności *PokerkalkActivity*.

Dodatkowo w projekcie wykorzystano zewnętrzne narzędzia:

- *Retrofit* i *Gson*: Służą do łączenia się z internetowym API (deckofcardsapi.com) i zamiany tekstu (JSON) pobranego z sieci na obiekty zrozumiałe dla Kotliny.
- *Coil*: biblioteka do ładowania obrazków. Bierze link (URL) do karty i wyświetla jej grafikę na ekranie telefonu.

2. Jak aplikacja przechowuje dane (Zmienne i Klasy)

Aplikacja przechowuje wszystkie informacje o rozdaniu w kilku prostych listach i specjalnych klasach pomocniczych. Można je podzielić na dwie grupy:

A. Dane z Internetu (Karty graficzne)

- *ApiCard*: Prosta klasa, która przechowuje informacje o pojedynczej karcie pobranej z sieci (np. jej kod "AS", link do obrazka, wartość i kolor).
- *allApiCards*: Główna lista, w której po uruchomieniu aplikacji lądują 52 karty pobrane z API.

B. Dane do Obliczeń (Logika gry)

Zwykły tekst z API trudno się sortuje matematycznie, dlatego program tłumaczy pobrane karty na specjalne typy wyliczeniowe (enum):

- Suit (Kolor): Reprezentuje 4 kolory w kartach: Kier, Karo, Trefl, Pik.
- Rank (Wartość): Przypisuje kartom liczby. Zwykła dwójka ma wartość 2, a As ma najwyższą wartość 14. To bardzo ułatwia sprawdzanie np. stritów.
- HandType: Słownik wszystkich układów w pokerze (od najwyższego ROYAL_FLUSH do najłabszego HIGH_CARD).

C. Pamięć obecnego stołu

Aby wiedzieć, co aktualnie dzieje się na ekranie, program używa list:

- *dealerHandApi*: Lista przechowująca od 0 do 5 kart leżących aktualnie na stole.
- *playerHandsApi*: Lista zawierająca "ręce" poszczególnych graczy (każdy gracz ma swoją własną listę na maksymalnie 2 karty).
- *buttonCardMap*: Sprytny słownik, który łączy konkretny przycisk na ekranie telefonu z kartą, która została w nim wyświetlona.

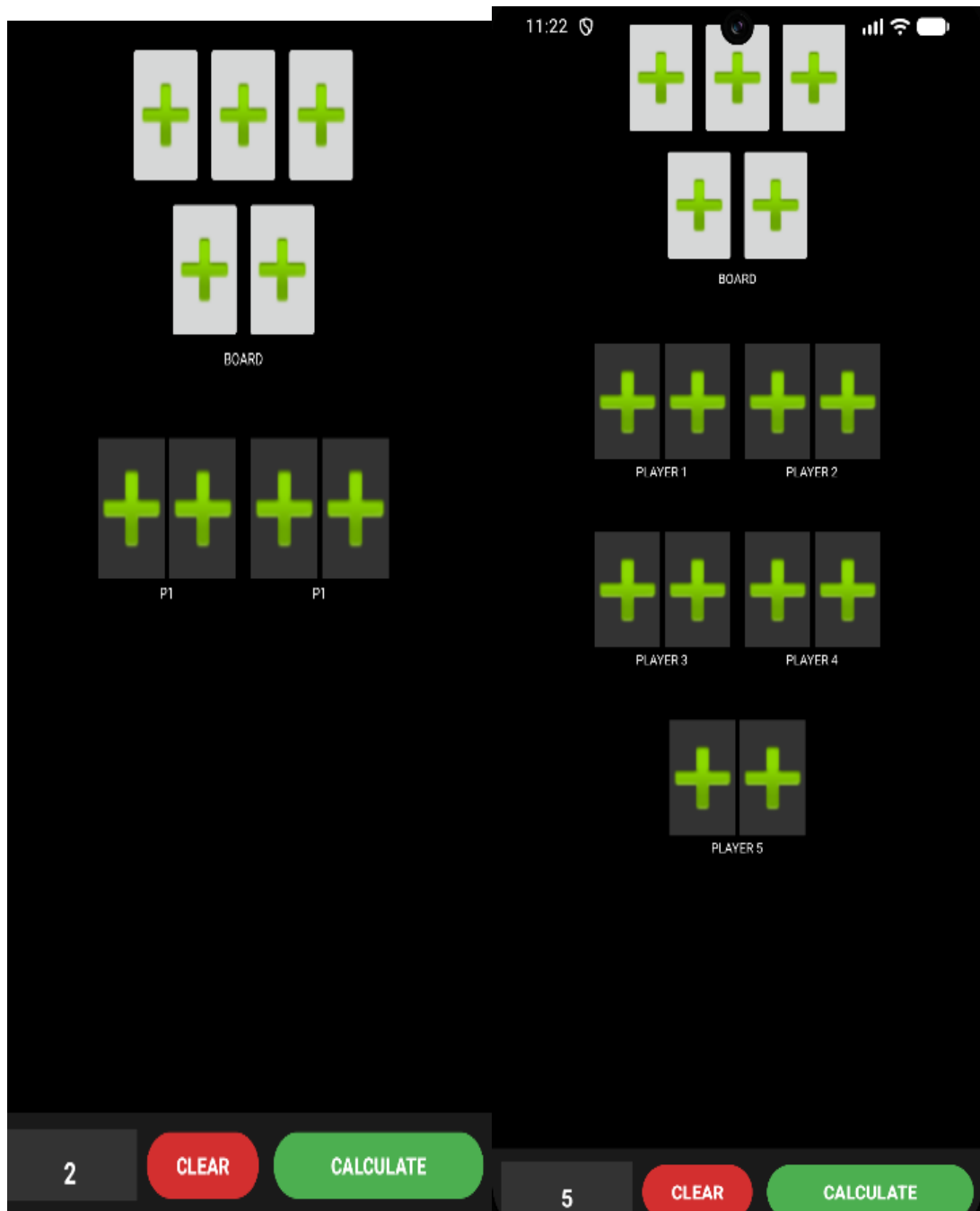
3. Interfejs użytkownika

Interfejs użytkownika jest zbudowany hierarchicznie i dynamicznie reaguje na liczbę graczy.

Układy (*Layouts*)

1. Główny ekran (*activity_pokerkalk.xml*): * Wykorzystuje *ConstraintLayout* jako główny kontener.

- Sekcja Dealera: Sztywny blok na górze z 5 przyciskami (3 na Flop, 1 na Turn, 1 na River).
 - Sekcja Graczy: Zapakowana w ScrollView, co gwarantuje poprawną obsługę aplikacji na mniejszych ekranach. Widoki graczy (od P1 do P5) są ułożone w parach za pomocą LinearLayout.
 - Bottom Bar: Dolny, zablokowany panel narzędziowy zawierający sterowanie liczbą graczy, przycisk resetu (Clear) i kalkulacji (Calculate).
2. Komponent Gracza (item_player.xml):
- Zreżywalny komponent dla każdego z 5 graczy (tag <include>).
 - Zawiera dwa miejsca na karty (card1, card2), nazwę gracza oraz ukryte domyślnie pole tekstowe winner_text, w którym pojawia się wynik procentowy na kolor złoty.



4. Komunikacja z Api

Funkcja `fetchDeckAndDrawCards()`

Zaraz po starcie aplikacji, program komunikuje się z serwerem kart.

1. Uruchamiana w przestrzeni `lifecycleScope.launch`, dzięki czemu UI nie zamarza podczas oczekiwania na internet.
2. Tworzona jest nowa talia: `api.getNewDeck()`.
3. Z nowej talii od razu losowane i pobierane są wszystkie 52 karty:
`api.drawCards(..., 52)`.

4. Karty te zapisywane są w pamięci lokalnej (allApiCards).
5. Obsługiwane są wyjątki try-catch w przypadku braku internetu, użytkownik otrzymuje komunikat "Błąd sieci!".

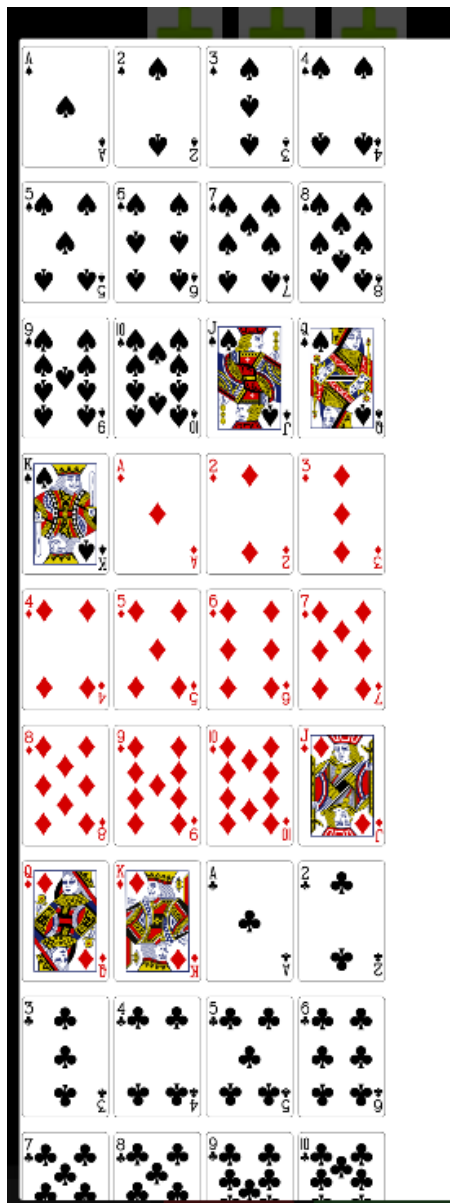
5. Mechanika Wyboru Kart

Aplikacja musi upewnić się, że jedna karta fizycznie nie wystąpi w dwóch miejscach jednocześnie.

Funkcja *showCardSelectionDialog()*

Gdy użytkownik kliknie pustą kartę, wywoływana jest ta funkcja.

1. Filtrowanie dostępnych kart: Algorytm pobiera wszystkie karty użyte na planszy (*usedCodes*), a następnie z puli 52 kart wyklucza te zajęte.
 - A. *Wyjątek*: Jeśli użytkownik klika na miejsce, w którym *już* jest karta i chce ją podmienić, ta konkretna karta jest nadal widoczna do wyboru.
2. Generowanie Widoku Dialogu: Kod dynamicznie (z poziomu Kotliny, bez pliku XML) tworzy *ScrollView* oraz *GridLayout* o 4 kolumnach. Do siatki dodawane są elementy *ImageView* z załadowanymi przez Coil grafikami kart.
3. Przypisanie i Podmiana: Po kliknięciu karty w dialogu:
 - A. Usuwana jest stara karta (jeśli jakaś była przypisana do tego przycisku).
 - B. Dodawana jest nowa karta do listy stołu (dla `playerIndex == -1`) lub odpowiedniej ręki gracza.
 - C. Grafika przycisku na głównym ekranie ulega aktualizacji.



6. Silnik Logiczny: Ewaluacja Układów Pokerowych

Jest to najbardziej skomplikowany funkcja aplikacji. Ponieważ każda karta z API ma typ string, przed oceną układu następuje jej tłumaczenie funkcjami *getRankFromApi()* oraz *getSuitFromApi()*.

6.1 Funkcja *evaluateHand()*

Funkcja przyjmuje listę 7 kart (2 z ręki + 5 ze stołu) i zwraca najsilniejszy możliwy układ (*EvaluatedHand*). Algorytm przebiega wieloetapowo:

1. Przygotowanie Danych:

- A. Karty są sortowane malejąco po wartości (value).

- B. Program szuka, czy występuje 5 kart w tym samym kolorze (potencjał na *Flush*).
 - C. Wyciąga same unikalne wartości kart do sprawdzenia *Strita*.
- 2. Sprawdzanie Strita (Straight):
 - A. Program iteruje po posortowanych unikalnych wartościach. Jeśli różnica między pierwszą a piątą kartą w oknie badawczym wynosi dokładnie 4 (np. $9 - 5 == 4$), oznacza to, że mamy strita.
 - B. Edge-case (As jako niska karta): Kod posiada specjalny warunek sprawdzający tzw. "Wheel", czyli układ 5, 4, 3, 2, As (gdzie As ma standardowo przypisaną najwyższą wartość 14).
- 3. Sprawdzanie Pokerów (Straight Flush / Royal Flush):
 - A. Jeśli wykryto i kolor, i strita, funkcja weryfikuje, czy karty tworzące kolor tworzą również strita. Jeśli tak, i układ zaczyna się od Asa (Wartość 14) zwracany jest *Royal Flush*. W innym wypadku *Straight Flush*.
- 4. Analiza Par, Trójek i Karet:
 - A. Karty grupowane są po ich wartości (`groupBy { it.value }`). Otrzymujemy np. grupę trzech Króli, grupę dwóch Piątek i pojedyncze karty.
 - B. Grupy sortowane są po liczebności (najpierw największe grupy), a w przypadku remisów - po wartości karty.
 - C. Następuje struktura warunkowa:
 - a. Grupa 4elementowa -> *Kareta (Four of a Kind)*.
 - b. Grupa 3elementowa + inna grupa min. 2-elementowa -> *Full House*.
 - c. Zwyczajny *Flush* (jeśli nie było układu wielokrotnego).
 - d. Zwyczajny *Straight* (jw).
 - e. Grupa 3-elementowa -> *Trójka (Three of a Kind)*.
 - f. Dwie oddzielne grupy 2-elementowe -> *Dwie Pary (Two Pair)*. Kod zadba o wybranie dwóch *najwyższych* par, jeśli gracz ma ich trzy.
 - g. Jedna grupa 2-elementowa -> *Para (One Pair)*.
 - h. Brak powyższych -> *Wysoka Karta (High Card)*.

6.2 Funkcja `compareHands()`

Dwie wyewaluowane ręce są porównywane. Jeśli typ układu (np. Kareta > Para) jest różny, wygrywa wyższy. Jeśli typ jest ten sam (np. Para vs Para), algorytm iteruje po wartości głównej układu (ranks), a na końcu porównuje karty poboczne (kickers). Funkcja oddaje 1 (A wygrywa), -1 (B wygrywa) lub 0 (Remis).

7. Algorytm Obliczania Szans (Symulacja *Monte Carlo*)

Dokładne wyliczenie szans w pokerze (szczególnie przy braku Flopa i wielu graczy) na bieżąco wymagałoby analizy milionów wariacji, co spowolniłoby urządzenie mobilne. Dlatego zastosowano wydajną metodę statystyczną *Monte Carlo*.

Funkcja *evaluateAndShowWinner()*

1. Czyszczenie i Walidacja: Przed obliczeniami ukrywane są stare wyniki procentowe. Program sprawdza, czy na stole lub w rękach graczy wybrana jest choć jedna karta. Jeśli nie blokuje wykonanie z powiadomieniem typu *Toast*.
2. Inicjalizacja Zmiennych: Przygotowywane są tablice wins (zwycięstwa) i draws (remisy) dla każdego gracza.
3. Pętla Symulacji (1000 iteracji):
 - a. Na każdą z 1000 iteracji tworzona jest "wirtualna kopia" (*simBoard*, *simHands*), będąca kopią aktualnego stanu na ekranie.
 - b. Obliczana jest *availableDeck* lista wszystkich pozostałych w talii, niewidocznych na stole kart. Następuje jej przetasowanie (*shuffled()*).
 - c. Rozdanie Wirtualne: Z przetasowanej talii dobierane są brakujące karty dla wszystkich aktywnych graczy (tak by każdy miał 2) oraz dobierane są karty na stół (tak by było ich 5).
 - d. Porównanie i Punktacja: Funkcja pętli ocenia każdą z 5-7 kartowych rąk wszystkich graczy korzystając z funkcji *compareHands()*. Jeśli dany gracz przebijie "najlepszego dotychczasowego", staje się nowym kandydatem na wygranego. W przypadku całkowitego remisu na koniec, flaga *isTie* powstrzymuje inkrementację licznika zwycięstw (aby suma wygranych była precyzyjna, można by podzielić punkt w przypadku remisu, co stanowiłoby opcję rozbudowy systemu).
4. Wyświetlenie Wyników: Liczba zwycięstw (*wins[i]*) dzielona jest przez liczbę iteracji (1000). Uzyskane prawdopodobieństwo matematyczne konwertowane jest na format procentowy i wyświetlane na ekranie (np. 55.4%).

Podsumowanie

PokerKalk to kompletna i w pełni funkcjonalna aplikacja mobilna, która w przystępny sposób rozwiązuje skomplikowany problem, jakim jest szacowanie prawdopodobieństwa wygranej w pokerze Texas Hold'em.

Link do chata(ogólnie to jest ten sam chat co użytkowa, ale w końcu nie wykorzystałem tego co mi wypłut chatgpt) <https://chatgpt.com/share/6a17663e-649c-83eb-bfc7-92ff12da2d80>